Build a Mini Coding Agent in JavaScript

One runnable stage at a time: a ReAct loop, nine tools, a workspace fence, long-term memory, nine lifecycle hooks and an offline test suite. This sample is **Stage 12**, the chapter where the loop becomes a library.

Craig West · craigwestai.com · *Stage 12 of 16*

CONTENTS

What is in this manual

About this sample 3

Stage 12: The ReAct loop 4

About the author 10

Page numbers refer to the folio at the foot of each page.

About this sample

Most coding-agent books start with a framework and explain the magic afterwards. This one starts with an empty folder. Every chapter adds one layer, runs it, proves it, and then explains the design decision behind it. By the end you have a command-line coding agent you can read in an afternoon, test with a fake model, and extend without guessing.

This sample contains a complete chapter from the middle of the book. **Stage 12** extracts the conversation loop into `agent.js` and turns it into a library with a step limit, a truncation guard, a heartbeat and a testable boundary. It is the chapter where the central idea of the book becomes literal code: *the loop is the agent*.

> **You do not need to write any code to follow the book.** Every stage ships as a complete, runnable snapshot in the companion repository. You read the finished code, run the commands, and tick the acceptance boxes. The work is in the experiments and the failures, not in typing files that already exist.

What you need for this chapter: Node.js 20 or newer and a terminal. The Stage 12 test suite uses a fake model, so it runs offline with no API key and no network. Only the last live command needs an OpenRouter key.

The companion repository: github.com/Python-Test-Engineer/build-mini-js-coder-book

---

Stage 12: The ReAct loop

Chapter snapshot: GitHub: `ch12/` — the complete runnable project after this stage.

What you can do at the end: run the whole agent as a testable library, with a step limit, a truncation guard and a heartbeat.

Plan

1. Extract the manual loop from `cli.js` into `runTask`.
2. Return outcomes rather than throwing task results.
3. Rebuild the system prompt for every task.
4. Implement the seven beats per step.
5. Add the truncation guard.
6. Add the heartbeat for slow model calls.
7. Guard direct execution of `agent.js`.

What is in `ch12`

You do not create or modify these files. The snapshot already contains everything listed below. From inside `ch12/`, run `npm install` once, read the files in the order shown, then run the commands under **Run it**.

New in this snapshot:

- `src/agent.js`
- `src/heartbeat.js`
- `test/agent.test.js`

Carried forward and changed:

- `src/cli.js` (replace the manual loop with `runTask`)

`runTask` signature:

```
runTask({
  task,
  workspace,
  sandbox = null,
  commandPolicy,
  maxSteps,
  chatFn,
  confirm,
  hooks = null,
  memory = null,
  history = [],
  log = console.log,
  heartbeatMs = 5000
})
```

Return `{ ok, answer, messages, steps }` or `{ ok: false, reason, messages }`.

How the code works:

- Rebuild the system prompt every task; filter old system messages out of `history`.
- `ctx = { workspace, sandbox, commandPolicy, memory, confirm }`; `confirm` defaults to deny.
- `hooks ?? new Hooks()` so every call site can call `hook.run` unconditionally.
- Direct execution prints the "agent.js is the loop, not the program" hint; never import `cli.js`.

Seven beats per step:

1. Blank line and `ui.step`.
2. Fire `PreModel`.
3. Start the heartbeat; call `chatFn`; on throw stop the heartbeat, print the error, fire `SessionEnd`, return `request_failed`; otherwise stop the heartbeat.
4. Extract the choice and message; fire `PostModel`.
5. Truncation guard: `finish_reason === 'length'` with no tool calls → append the assistant, append a user message `"You were cut off by the length limit. Continue with smaller tool calls."`, warn, `continue`. Never dispatch truncated calls.
6. Append `sanitizeAssistant`; print `ui.think` if there is content.
7. No calls → print the answer, fire `Stop` and `SessionEnd`, return `ok`. Calls → for each: print the tool, fire `PreToolUse`, block or run, print the result, fire `PostToolUse`, append exactly one tool message.

After `maxSteps`: print an error, fire `StepLimit` then `SessionEnd`, return `max_steps`.

Entry guards:

- `SessionStart` or `UserPromptSubmit` blocked → return `blocked_by_hook`.
- A rewrite replaces `messages[taskIndex]`.

`heartbeat(log, everyMs)` returns `{ stop }`; it prints `… still waiting for the model (<s>s)` and calls `timer.unref?.()`.

Run it

```
cd ch12
npm install
node --test test/agent.test.js
node src/agent.js
node src/cli.js "Run node --version and tell me the result"
```

Expected: the agent test suite runs the loop with a fake model and no network; `node src/agent.js` prints the hint; the live task still works through the new library.

Prove it

- [ ] Tools run, then the loop stops on a plain answer.
- [ ] A failing tool result is fed back so the model can correct.
- [ ] The loop is capped at `maxSteps`.
- [ ] A truncated reply is not dispatched; the model is asked to continue smaller.
- [ ] The heartbeat prints waiting lines during slow calls.

- [ ] Blocked hooks produce the right exit path.
- [ ] The step limit fires `StepLimit` then `SessionEnd`.
- [ ] A follow-up task sees previous conversation but a fresh system prompt.
- [ ] Direct execution prints the hint and does not import the CLI.

If it goes wrong

- **The system prompt appears twice** — filter old system messages from `history` and rebuild once.
- **A truncated call is dispatched** — check `finish_reason` before dispatching.
- **The process stays alive after a run** — `unref` the heartbeat timer.
- **The loop throws task outcomes** — return a reason; the CLI maps it to an exit code.

What this stage is really about

Stage 12 formalises the loop as a library: `runTask`. It is the chapter where "the loop is the agent" becomes literal code you can read in one sitting.

The extraction is deliberately late. By now you have built the loop by hand in `cli.js`, added tools to it, and wired hooks into it. You know what belongs there. Extracting earlier would produce a library nobody understands.

The signature

```
export async function runTask({
  task,
  workspace,
  sandbox = null,
  commandPolicy,
  maxSteps,
  chatFn,
  confirm,
  hooks = null,
  memory = null,
  history = [],
  log = console.log,
  heartbeatMs = 5000,
}) { /* ... */ }
```

Return shapes:

```
{ ok: true,  answer, messages, steps }
{ ok: false, reason, messages }
```

Why an object rather than throwing? Because a task outcome is data the CLI can inspect and turn into an exit code. `request_failed` and `max_steps` are expected outcomes, not exceptional ones.

The golden rule

`agent.js` is a library. It knows nothing about flags, colours, `.env`, or `process.exit`. The CLI owns all process concerns. Tests can run the loop with a fake model and no network.

This is not purity for its own sake. It is what makes the loop testable:

- pass a fake `chatFn`;
- pass a temp `workspace`;
- pass a fake `confirm`;
- pass `log: () => {}`;
- assert on the returned `messages` and `ok`.

No network, no API key, no terminal. The entire Stage 15 test suite depends on this boundary.

The seven beats per step

For `step = 1..maxSteps`:

1. **Print a step heading.** `ui.step(step, maxSteps)`.
2. **Fire** `PreModel` with `{ step, maxSteps, messages, tools: TOOL_SCHEMAS }`.
3. **Start the heartbeat and call the model.** - On throw: stop the heartbeat, print `ui.error`, fire `SessionEnd`, return `{ ok: false, reason: 'request_failed', messages }`. - On success: stop the heartbeat.
4. **Extract the choice and message.** Fire `PostModel` with `{ step, message, finishReason }`.
5. **Truncation guard.** If `finish_reason === 'length'` and there are no tool calls: - append the assistant message; - append a `user` message: `"You were cut off by the length limit. Continue with smaller tool calls."`; - print a warning; - `continue` to the next step. Never dispatch truncated tool calls.
6. **Append** `sanitizeAssistant(message)`**.** If `content` is non-empty, print `ui.think(...)`.
7. **Dispatch tool calls or finish.** - **No calls:** print the answer, fire `Stop`, fire `SessionEnd`, return `{ ok: true, answer, messages, steps: step }`. - **Calls:** for each in order:
   1. read `name` and `rawArgs`;
   2. print `ui.tool(name, rawArgs)`;
   3. fire `PreToolUse`; if blocked, observation = `DENIED: a PreToolUse hook blocked this call (<reason>). Try another approach.`; otherwise `await runTool(name, rawArgs, ctx)`;
   4. print `ui.result(result)`;
   5. fire `PostToolUse`; if `replace !== null`, use the replacement;
   6. append exactly one `{ role: 'tool', tool_call_id: call.id, content: String(post.replace ?? result) }`.

After the loop exhausts `maxSteps`:

- print `ui.error('stopped: hit the <maxSteps>-step limit')`;
- fire `StepLimit`, then `SessionEnd`;
- return `{ ok: false, reason: 'max_steps', messages }`.

The truncation guard is the interesting one

A cut-off reply often ends mid-JSON:

```
{"path": "hello.js", "cont
```

Dispatching that would produce `ERROR: could not parse arguments` for a tool call the model never finished, and the model would spend a step apologising for a bug it did not have.

Refusing to dispatch it is the difference between a harness that respects the model and one that argues with it. The corrective message is a `user` turn, not a system change: the model is told what happened and asked to continue smaller.

Why the system prompt is rebuilt

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
```

Two details are the whole trick:

1. **The system message is rebuilt**, so a memory saved by the previous task is already in the prompt the next task sends.
2. **The history is filtered** to remove any old system message, so there is exactly one.

If you carried the old system message forward, memory would be stale and the prompt would slowly accumulate duplicates.

The hook default

```
const hook = hooks ?? new Hooks();
```

Why not `if (hooks) await hooks.run(...)` at every call site? Because that would litter the loop with conditionals. A null-object default lets every call site call `hook.run(...)` unconditionally. This is a small pattern worth learning: **make the absent case a working object, not a special case.**

Entry guard: `agent.js` is not the program

Running `node src/agent.js` should print:

```
agent.js is the loop, not the program.
Run the CLI instead:  node src/cli.js --help
```

and exit. `agent.js` must not import `cli.js`. The dependency arrow points one way.

The heartbeat

```
export function heartbeat(log, everyMs = 5000) {
  const started = Date.now();
  const timer = setInterval(() => {
    const seconds = Math.round((Date.now() - started) / 1000);
    log(ui.waiting(`  … still waiting for the model (${seconds}s)`));
  }, everyMs);
  timer.unref?.();
  return { stop: () => clearInterval(timer) };
}
```

Why `unref()`? A timer that holds the event loop open would turn a clean exit into a program that hangs for five more seconds. `unref` says "this timer must not keep the process alive on its own".

Why arm it just before the call and stop it just after, including on the failure path? Because a leaked timer prints "still waiting" after the run is over, which is worse than no heartbeat.

Checkpoint

1. Why is `runTask` a library rather than part of `cli.js`?

A. So it can be tested and read without the CLI; the dependency arrow points one way.

2. Why are `request_failed` and `max_steps` returned rather than thrown?

A. They are expected task outcomes the CLI turns into exit codes; throwing would lose the message array.

3. Walk through the seven beats from memory. Which beat fires which hook?

A. Step heading → `PreModel`; model call; `PostModel`; truncation guard; append/sanitise; `Stop` / `PreToolUse`; `PostToolUse`; `StepLimit` / `SessionEnd`.

4. Why is a truncated tool call not dispatched?

A. A truncated call is half a JSON object; dispatching it produces a confusing error for a call the model never finished.

5. Why is the system prompt rebuilt for every task?

A. Memory may have changed, and a carried system message would be stale or duplicated.

6. What does `hooks ?? new Hooks()` buy the call sites?

A. It makes the absent case a working object, so call sites need no conditionals.

7. Why does the heartbeat call `unref()`?

A. So the timer never keeps the process alive after the run is finished.

About the author

Craig West writes and teaches about applied AI and agent engineering. He is the author of *mini-js-coding-harness* and other practical courses that build AI systems from first principles rather than hiding them behind a framework.

You can find his writing, courses and contact details at **craigwestai.com**.

The course files, the full manuscript and the student companions are public:

- **Book and chapters:** github.com/Python-Test-Engineer/build-mini-js-coder-book
- **Student Q&A:** qa.html
- **Code reference:** function-reference.md

This is a sample chapter. The full book covers Stages 0–16, the teacher's handbook, the student workbook, the deep dives and the reference appendices.